6-4 · CDC(Debezium) 실시간 캡처 · 실습 2.5H

커넥터 구축과 변경 이벤트 분석

커넥터를 등록하고 네 가지 op 를 눈으로

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

2.5시간 뒤에는

① mysql·connect 를 더한 compose 로 다시 쓰고 binlog 설정을 확인한다

② REPLICATION 권한의 debezium 계정과 orders 표를 만든다

③ REST API 로 mysql-cdc 커넥터를 등록해 RUNNING 을 확인한다

④ op = r · c · u · d 네 가지를 파이썬으로 읽어 정리한다

일부러 decimal 설정 · 툼스톤 처리를 빼 보고 무엇이 깨지는지 봅니다

실습 흐름

일곱 단계 — compose 다시 쓰기부터 파이썬 정리까지

~/data-eng/week6 에서 시작 — kafka 는 그대로, compose 파일 전체를 다시 씁니다

1. 스택 추가	2. 계정·표	3. 커넥터 등록	4·5. 이벤트 관찰	6·7. 파이썬 정리
compose 다시 쓰기 binlog 확인	debezium 계정 orders 2행	POST /connectors RUNNING 확인	op = r 스냅숏 c · u · d	cdc_reader.py 앱 수정 없음

1단계

compose 다시 쓰기와 binlog 확인

compose 다시 쓰기	cat > docker-compose.yml << 'EOF' … EOF
서비스	kafka · mysql:8.0 · debezium/connect:2.7.3.Final
기동	docker compose up -d → kafka Running · 둘 Started
binlog 확인	log_bin=ON · binlog_format=ROW
Connect 응답	curl … :8083/ → {"version":"3.7.0", …}
플러그인	…/connector-plugins → MySqlConnector 포함 10줄

MySQL 은 약 20초 준비(그 전엔 ERROR 2002) · Connect 는 15초쯤(그 사이 HTTP 404) — 잠시 뒤 다시

2단계

CDC 계정과 orders 표 — Debezium 은 읽기만

1. **debezium 계정 — REPLICATION 권한**
   REPLICATION SLAVE·CLIENT 가 binlog 를 읽는 권한 — 쓰기 권한은 주지 않는다

2. **orders 표 — id · customer · amount · status**
   status 기본값 'CREATED' — user1 12000 · user2 35000 두 행 INSERT

3. **확인**
   SELECT * FROM orders; 로 2행이 조회되면 성공

3306 이 겹치면

docker stop

de-mysql

3단계

register-mysql.json — 일곱 설정과 REST 등록

설정 키	값	뜻
connector.class	…mysql.MySqlConnector	Debezium MySQL 커넥터
database.hostname	mysql	컨테이너 이름으로 접속
database.user / password	debezium / dbz	2단계에서 만든 계정
topic.prefix	shop	토픽 이름 맨 앞
database.include.list	shop	캡처할 DB
decimal.handling.mode	string	금액을 "12000.00" 으로
schema.history…servers	kafka:29092	내부 리스너(6-2)

POST 뒤 status 에서 connector · tasks[0] 모두 RUNNING — 토픽 셋: shop · shop.shop.orders · schema-changes.shop

3단계 — 일부러 빼 보기

decimal.handling.mode 를 빼면 금액이 깨진다

빼면

"amount": "Ek+A" — 알아볼 수 없는 글자

- 정밀도를 지키려고 숫자를 바이트로 보낸다
- JSON 에는 그 바이트가 base64 로 찍힌다
- 6-5 MinIO 파일에도 그대로 담긴다

VS

"string" 으로 두면

"amount": "12000.00" — 사람이 읽는 값

- after 의 amount 를 바로 읽는다
- 7단계 13200.00 · 10890.00 도 이 형태
- 오늘 JSON 에는 넣어 둔 채로 등록

4·5단계

네 가지 op 를 터미널 두 개로

실행한 SQL	op	before	after
커넥터 등록 시 이미 있던 2행	r	null	현재 행
INSERT ('user3', 9900)	c	null	넣은 행
UPDATE status='PAID' WHERE id=1	u	status=CREATED	status=PAID
DELETE WHERE id=2	d	지워진 행	null
DELETE 뒤 한 줄 더	—	툼스톤	null(값 없음)

터미널 1 에 consumer 를 켜 둔 채 터미널 2 에서 한 줄씩 — 스냅숏 줄은 3KB 쯤, 끝 쪽 payload 에서 op 를 찾는다

6·7단계

cdc_reader.py — 그리고 「앱을 고치지 않는다」

1. **Consumer 로 shop.shop.orders 구독**
   group.id cdc-reader · auto.offset.reset earliest — 스냅숏부터 다시 읽는다

2. **payload 에서 op·before·after 만 찍는다**
   OP = {r: 스냅숏, c: INSERT, u: UPDATE, d: DELETE} · 값이 없으면 [툼스톤] 으로

3. **7단계 — 평범한 SQL 여러 건**
   다시 켜면 INSERT 2 · UPDATE 4 — 앞에서 읽은 것은 안 나온다

실행

python

cdc_reader.py

6단계 — 일부러 빼 보기

툼스톤 처리 두 줄을 빼면 멈춘다

빼면

AttributeError: 'NoneType' ... 'decode'

- DELETE 뒤 값이 없는 메시지가 온다
- msg.value() 가 None 이라 decode 에서 멈춘다
- 받는 쪽은 빈 메시지도 견뎌야 한다

VS

처리하면

[툼스톤] id=2 — 값이 없는 메시지

- if msg.value() is None: 출력 뒤 continue
- 툼스톤 = 「이 키는 지워졌다」 표시
- 6-5 Sink 도 이 메시지를 따로 다룬다

확인 포인트

이 2.5시간이 끝날 때 확인할 것

1. log_bin=ON · binlog_format=ROW · curl 8083 에 version 3.7.0
2. SELECT * FROM orders 에 2행
3. status 의 connector.state·tasks[0].state 가 RUNNING
4. 토픽 셋 — shop · shop.shop.orders · schema-changes.shop
5. op=r → c · u(CREATED→PAID) · d(after=null) · 툼스톤
6. cdc_reader 에 네 종류 + 툼스톤 · 7단계는 INSERT 2 · UPDATE 4

6-4 · 실습 2.5H 마무리

다음은 정리·회고 (0.5H)

Connect REST API 를 한 장으로 정리하고, 6-5 적재를 위해 커넥터와 토픽이 살아 있는지 확인합니다

- ✓ **REST API 여덟 명령 요약**
- ✓ **체크리스트 자가 점검**
- ✓ **자주 겪는 오류 네 가지**
- ✓ **6-5 준비: 커넥터·토픽 상태**

가져갈 것

UPDATE 이벤트의 before·after status 값